МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждения образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
Кафедра программной инженерии
Специальность 6-05-0612-01 Программная инженерия
Направление специальности 6-05-0612-01 Программная инженерия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
КУРСОВОГО ПРОЕКТ:

по дисциплине «Базы данных»
Тема «Реализация базы данных интернет-банка с применением технологий шифрования и маскирования данных в СУБД Oracle»

Исполнитель
студент (ка) 2 курса группы 9 Антипов Алексей Романович
(Ф.И.О.)

Руководитель работы преподаватель-стажёр Заянковский Д.В
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой
Председатель Нистюк О.А.
(подпись)

Минск 2026

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»
Факультет информационных технологий
Кафедра информационных систем и технологий

УТВЕРЖДАЮ
Заведующий кафедрой
_____ Е.А. Блинова
подпись инициалы и фамилия

«____» _____2026 г.

ЗАДАНИЕ
к курсовой работе
по дисциплине «Базы данных»

Специальность 6-05-0612-01 Программная инженерия (профилизация Программное обеспечение информационных технологий)

Курс: 3 Группа: 9

Студент: Антипов Алексей Романович
Тема: «Реализация базы данных интернет-банка с применением технологий шифрования и маскирования данных в СУБД Oracle»

1. Срок сдачи студентом законченной работы: «15» декабря 2026 г.

2. Исходные данные к курсовой работе:

2.1. Функционально должны быть выполнены следующие задачи:

Роль Администратор:
Добавление, изменение и блокировка учетных записей сотрудников банка;
Добавление, изменение и блокировка учетных записей клиентов;
Назначение ролей и прав доступа к данным;
Формирование отчетов по клиентам, счетам и банковским операциям.

Роль Сотрудник банка:
Открытие и закрытие банковских счетов, изменение их состояния;
Выпуск и блокировка банковских карт, изменение их состояния;
Контроль доступности денежных средств и соблюдения установленных лимитов операций.

Роль Клиент:
Регистрация и авторизация;
Выполнение переводов между банковскими счетами;
Выполнение платежных операций;
Просмотр истории операций и формирование выписок по счетам.

Для всех ролей:
Просмотр данных о клиентах, счетах, картах и операциях в соответствии с назначенными правами доступа;
Маскирование конфиденциальных данных в зависимости от роли пользователя;
Шифрование и расшифрование конфиденциальных данных клиентов средствами СУБД Oracle.

2.2. Требования.

База данных должна быть реализована в СУБД Oracle.

Доступ к данным должен осуществляться только через соответствующие процедуры.

Количество объектов БД (таблиц, представлений, индексов, пользователей и пр.) регламентируется задачей.

Должен быть проведен импорт данных из JSON файлов, экспорт данных в формат JSON.

Необходимо протестировать производительность базы данных на таблице, содержащей не менее 100 000 строк, и внести изменения в структуру в случае необходимости. Необходимо проанализировать планы запросов к таблице.

Применить технологию базы данных согласно выбранной теме: подробно описать применяемые системные пакеты, утилиты или технологии; показать применение указанной технологии в базе данных.

Листинги проекта должны содержать комментарии.

3. Содержание расчетно-пояснительной записки

Введение
Постановка задачи
Проектирование базы данных.
Разработка объектов базы данных
Описание процедур импорта и экспорта
Тестирование производительности
Описание технологии и ее применения в базе данных
Краткое описание приложения для демонстрации
Руководство пользователя
Заключение
Список используемых источников
Приложения

4. Форма представления, выполненного курсовой работы:

Пояснительная записка оформляется в MS Word.

Оформление пояснительной записки должно соответствовать требованиям к оформлению пояснительной записки для курсовых работ.

Листинги всех скриптов представляются в приложении.

К записке необходимо приложить DVD-диск, который должен содержать пояснительную записку, листинги и файлы базы данных.

Календарный план

№ п/п	Наименование этапов курсовой работы	Срок выполнения этапов работы	Примечание
1	Введение	21.09.2026	
2	Аналитический обзор литературы по теме работы	28.09.2026	
3	Изучение требований, определение вариантов использования	05.10.2026	
4	Анализ и проектирование модели базы данных. Описание информационных объектов и ограничений целостности.	16.10.2026	
5	Создание необходимых объектов	24.10.2026	
6	Импорт и экспорт данных	08.11.2026	
7	Описание используемой технологии	14.11.2026	
8	Тестирование производительности	21.11.2026	
9	Оформление пояснительной записки	05.12.2026	
10	Сдача курсовой работы	15.12.2026	

5. Дата выдачи задания «__» _____ 2026 г.

Руководитель _____ Д.В. Заянковский
(подпись)

Задание принял к исполнению _____
(дата и подпись студента)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1 Постановка задачи 6
1.1 Назначение и границы разрабатываемой базы данных 6
1.2 Функциональные требования и роли пользователей 6
1.3 Требования к целостности и организации доступа 7
1.4 Варианты использования 9
2 Проектирование базы данных 10
2.1 Информационные объекты предметной области 10
2.2 Логическая схема и связи между сущностями 11
2.3 Нормализация и выбор типов данных 12
2.4 Структура основных таблиц 13
2.5 Правила целостности и жизненный цикл объектов 14
3 Разработка объектов базы данных 16
3.1 Состав схемы и порядок создания объектов 16
3.2 Ограничения целостности и индексы 16
3.3 Представления и процедурный интерфейс 17
3.4 Транзакционная обработка переводов и платежей 19
3.5 Шифрование и маскирование конфиденциальных данных 20
3.6 Роли Oracle и контроль выполнения процедур 21
3.7 Аудит и подготовка к последующим этапам 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 23

ВВЕДЕНИЕ

Интернет-банк обеспечивает дистанционное обслуживание клиентов и объединяет операции с банковскими счетами, картами и платежами. Основой такого приложения является база данных, в которой фиксируются владельцы счетов, доступные остатки и результаты денежных операций. Ошибка при изменении остатка или определении прав пользователя может привести к нарушению целостности финансовых данных и раскрытию конфиденциальной информации.

Тема курсового проекта — «Реализация базы данных интернет-банка с применением технологий шифрования и маскирования данных в СУБД Oracle». Актуальность работы определяется необходимостью совместного решения двух задач: согласованного выполнения банковских операций и ограничения доступа к сведениям о клиентах. Шифрование защищает сохраняемые значения, а маскирование определяет объем информации, который получает пользователь при обращении к базе данных. Эти механизмы должны применяться совместно с проверкой полномочий и владельца запрашиваемой записи.

Цель курсового проекта — разработать реляционную базу данных интернет-банка в СУБД Oracle, обеспечивающую учет клиентов, сотрудников, счетов, карт и операций, а также защиту конфиденциальных сведений средствами шифрования, маскирования и разграничения доступа.

Объект исследования — хранение и обработка данных дистанционного банковского обслуживания. Предмет исследования — структура реляционной базы данных, транзакционная обработка переводов и платежей, программные интерфейсы PL/SQL и способы защиты конфиденциальных полей.

Для достижения поставленной цели необходимо решить следующие задачи:

— определить функции администратора, сотрудника банка и клиента, а также правила доступа к данным;

— спроектировать сущности, связи, ключи и ограничения целостности реляционной модели;

— разработать структуру таблиц, индексов, представлений и процедурного интерфейса;

— предусмотреть атомарное выполнение денежных операций с проверкой остатков и лимитов;

— организовать шифрование конфиденциальных полей и формирование представления данных с учетом роли пользователя;

— предусмотреть импорт и экспорт JSON, аудит действий и последующую проверку производительности на объеме не менее 100 000 записей.

Практический результат работы должен представлять собой базу данных Oracle с документированной структурой и интерфейсом хранимых процедур для демонстрационного приложения. Переводы и платежи рассматриваются только между зарегистрированными в системе счетами.

1 Постановка задачи

1.1 Назначение и границы разрабатываемой базы данных

База данных предназначена для централизованного хранения сведений интернет-банка и выполнения операций от имени пользователей с установленными правами. Обслуживаемые объекты включают учетные записи, клиентские и служебные профили, банковские счета, карты, лимиты и историю денежных операций. Система должна сохранять историю обслуживания при блокировке пользователя или закрытии счета.

Для расчетов используются валюты BYN, USD и EUR. Денежные значения хранятся с двумя десятичными разрядами. Перевод между счетами разных валют отклоняется, поскольку механизм конвертации не входит в границы проекта. Карта является платежным инструментом, связанным со счетом, и не имеет самостоятельного остатка. Фактическая эмиссия карты в платежной системе не моделируется.

Принимается модель персональных учетных записей Oracle: зарегистрированный пользователь приложения подключается к СУБД от собственного имени. Соответствие между учетной записью Oracle и прикладным профилем хранится в BANK_USERS. Аутентификация осуществляется средствами Oracle; пароль не дублируется в прикладных таблицах. Проверка бизнес-полномочий выполняется при каждом вызове процедур.

1.2 Функциональные требования и роли пользователей

Администратор управляет учетными записями клиентов и сотрудников, назначает прикладные роли и формирует сводные отчеты. Бизнес-роль администратора не является ролью DBA и не предоставляет полномочий владельца схемы. Для администратора предусматриваются следующие функции:

— создание и изменение прикладных учетных записей клиентов и сотрудников;

— блокировка и восстановление доступа пользователей с сохранением их данных;

— назначение и отзыв ролей в пределах установленного набора полномочий;

— получение отчетов по клиентам, состояниям счетов и проведенным банковским операциям.

Сотрудник банка выполняет операции обслуживания счетов и карт. В его полномочия входят:

— открытие и закрытие счетов, изменение их состояния при соблюдении установленных правил;

— выпуск, блокировка и изменение состояния банковских карт;

— просмотр доступных остатков и установленных разовых и суточных лимитов.

Клиент получает доступ к собственным счетам и связанным с ними данным. Для клиента предусматриваются:

— регистрация и последующая аутентификация;

— переводы между своими счетами и на счета других клиентов в той же валюте;

— платежи на зарегистрированные в системе счета с указанием назначения;

— просмотр истории расходных и приходных операций и получение выписки за выбранный период.

Просмотр сведений о клиентах, счетах, картах и операциях выполняется в пределах полномочий вызывающего пользователя. Принадлежность данных проверяется в СУБД, а не только в интерфейсе приложения. Передача чужого идентификатора счета в параметрах процедуры не должна давать клиенту доступ к соответствующим сведениям.

Шифрование и расшифрование выполняются внутренними подпрограммами базы данных. Пользователь получает результат прикладной операции, а не универсальное право расшифровать произвольное поле. Объем раскрываемых данных определяется назначением процедуры, ролью и принадлежностью записи; номер карты выводится только в маскированном виде.

1.3 Требования к целостности и организации доступа

Доступ демонстрационного приложения к данным должен осуществляться исключительно через публичные процедуры PL/SQL. Изменения выполняются параметризованными вызовами; результаты чтения возвращаются через SYS_REFCURSOR либо CLOB. Прямые права SELECT, INSERT, UPDATE и DELETE на прикладные таблицы и внутренние представления пользовательским ролям не выдаются.

При проектировании устанавливаются следующие обязательные правила:

— сумма денежной операции должна быть положительной, а счета списания и зачисления — различными и открытыми;

— инициатором списания является клиент — владелец расходного счета; блокировка учетной записи запрещает дальнейшие обращения к прикладному API;

— текущий остаток расходного счета не может становиться отрицательным; учитываются разовый и суточный лимиты;

— списание, зачисление, регистрация операции и запись успешного аудита должны составлять одну транзакцию;

— повтор одного и того же запроса не должен приводить к повторному списанию;

— закрытие счета допускается при нулевом остатке и отсутствии действующих карт; закрытые объекты и проведенные операции физически не удаляются.

Сведения о документе клиента и полном номере карты должны храниться в зашифрованном виде. Ключи и исходные конфиденциальные значения не должны включаться в прикладные журналы, тексты исключений и файлы экспорта общего назначения. Для шифрования предусматривается DBMS_CRYPTO; алгоритмы и правила применения рассматриваются в разделе разработки объектов [1].

База данных должна поддерживать прием JSON-документов и формирование JSON-результатов. Для импорта предусматривается проверка структуры, обязательных полей и ограничений; импорт денежных операций не должен обходить проверку остатков и лимитов. Для последующего испытания производительности выбирается BANK_OPERATIONS с объемом не менее 100 000 строк. Сравнение планов и времени запросов выполняется на одном наборе данных до и после изменения индексов; численные результаты фиксируются после проведения испытаний.

Для перевода основной сценарий включает выбор расходного и приходного счетов, ввод суммы и назначения, проверку полномочий, состояния счетов, остатка и лимитов, проведение операции и выдачу ее идентификатора. При любой ошибке проверки денежные изменения не фиксируются. Просмотр истории включает отбор по собственному счету и периоду; формирование выписки использует тот же набор операций.

1.4 Варианты использования

На рисунке 1.1 представлены варианты использования для трех прикладных ролей. Обобщенный актор «Пользователь» отражает общие функции входа и просмотра разрешенных данных; специализации наследуют эти функции. Регистрация является отдельным действием до аутентификации. Шифрование и маскирование выполняются внутри соответствующих сценариев.

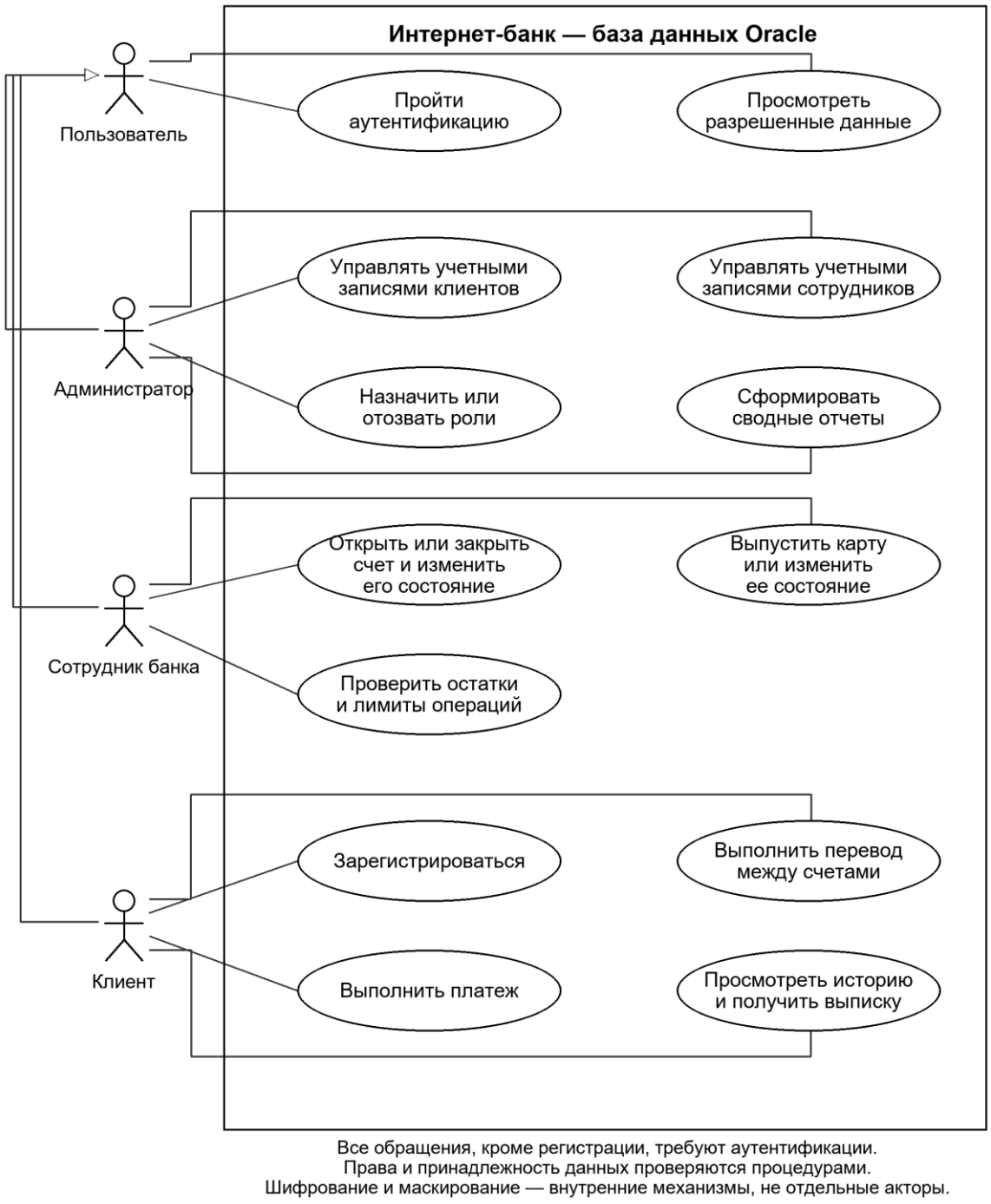


Рисунок 1.1 — Диаграмма вариантов использования интернет-банка

2 Проектирование базы данных

2.1 Информационные объекты предметной области

Учетные записи и личные профили разделены. BANK_USERS идентифицирует подключение и его состояние, BANK_CLIENTS содержит конфиденциальный профиль клиента, BANK_EMPLOYEES — профиль сотрудника. Параметры входа не дублируются в профилях, роли назначаются независимо от них.

Счет принадлежит одному клиенту и ведется в одной валюте. У счета может быть несколько карт, однако все они используют общий остаток счета. Информация о выполненной операции содержит два счета: debit_account_id обозначает счет списания, credit_account_id — счет зачисления. Отдельная таблица платежей не создается: тип операции задается справочником BANK_OPERATION_TYPES, а платеж дополнительно требует непустого назначения.

В структуре предусмотрено 12 таблиц. Их назначение приведено в таблице 2.1. Поля связей и остальные атрибуты отражены на логической схеме; внешние ключи ссылаются на первичные либо уникальные ключи родительских таблиц.

Таблица 2.1 — Состав таблиц базы данных

Таблица	Назначение
BANK_USERS	Учетная запись Oracle, логин, блокировка и дата регистрации. Пароль в таблице не хранится.
BANK_ROLES	Справочник ролей ADMIN, EMPLOYEE и CLIENT.
BANK_USER_ROLES	Назначения ролей; составной первичный ключ (user_id, role_id).
BANK_CLIENTS	Зашифрованный профиль клиента, HMAC документа и версия ключей.
BANK_EMPLOYEES	Учетная запись, зашифрованные Ф.И.О. и должность сотрудника.
BANK_CURRENCIES	Справочник валют BYN, USD и EUR.
BANK_ACCOUNTS	Владелец, валюта, номер, состояние, остатки и даты обслуживания счета.
BANK_CARDS	Связанный счет, шифротекст и HMAC PAN, последние четыре цифры, срок и состояние.
BANK_OPERATION_TYPES	Справочник типов TRANSFER и PAYMENT.
BANK_OPERATIONS	Инициатор, два счета, сумма, назначение, идентификатор запроса и время проведения.
BANK_ACCOUNT_LIMITS	Разовый и суточный лимиты расходных операций счета.
BANK_AUDIT_LOG	Автор, действие, объект, время и результат обращения без конфиденциальных значений.

2.2 Логическая схема и связи между сущностями

Логическая схема представлена на рисунке 2.1. Обозначения PK, FK и UK соответствуют первичному, внешнему и уникальному ключам. Числа на концах связей задают допустимую кратность. Для BANK_OPERATIONS звездочкой отмечен компонент составного уникального ограничения (initiator_user_id, request_id), а не самостоятельная уникальность request_id.

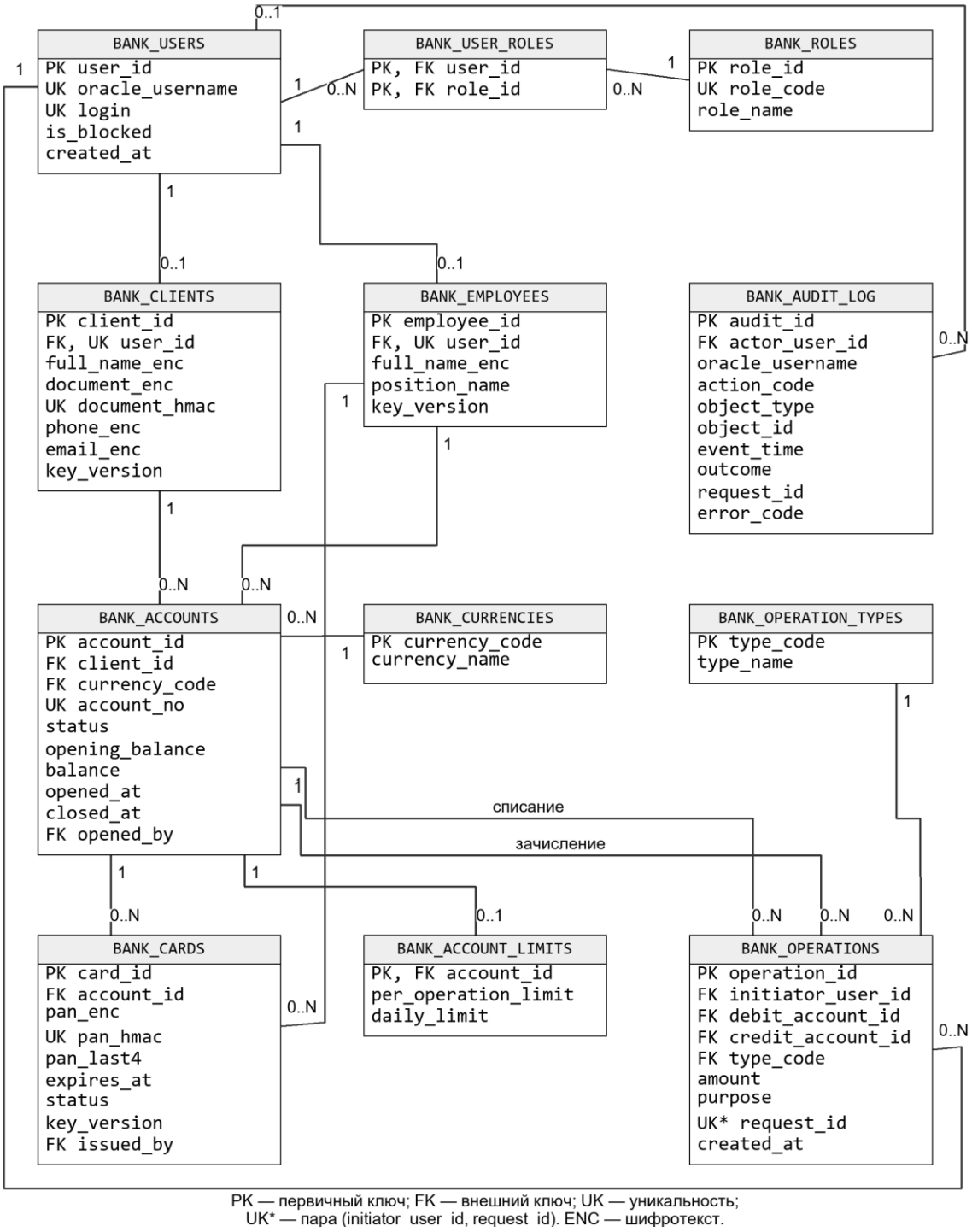


Рисунок 2.1 — Логическая схема базы данных интернет-банка

Связь пользователей и ролей имеет кратность «многие ко многим» и разрешается таблицей BANK_USER_ROLES. Каждая строка этой таблицы ссылается на одного пользователя и одну роль; повторное назначение той же пары исключается первичным ключом. Клиентский и служебный профили связаны с учетной записью отношением «один к нулю или одному» за счет уникальности user_id.

Один клиент может иметь несколько счетов; один счет — несколько карт и множество операций. С BANK_OPERATIONS установлены две самостоятельные связи со счетами: для списания и для зачисления. При построении истории счета должны учитываться обе связи. Валюта операции определяется валютой связанных счетов; отдельное поле валюты в BANK_OPERATIONS не дублируется.

BANK_ACCOUNT_LIMITS использует account_id одновременно как первичный и внешний ключ. На уровне ссылочной целостности это допускает отсутствие строки лимитов, поэтому процедура открытия счета создает счет и его лимиты совместно; проведение операции при отсутствии лимитов отклоняется. Ссылки opened_by и issued_by указывают на сотрудника, выполнившего открытие счета или выпуск карты.

В журнале аудита actor_user_id допускает NULL для события, которому еще не соответствует подтвержденный прикладной пользователь. Имя сессии Oracle сохраняется отдельно. Пара object_type и object_id описывает объект разных типов и не объявляется внешним ключом на одну конкретную таблицу. Прикладные профили, счета и журнал не удаляются каскадно.

2.3 Нормализация и выбор типов данных

Атрибуты таблиц содержат одиночные значения. Перечень ролей и набор карт не записываются в строковые списки: для них используются отдельные отношения. Это обеспечивает первую нормальную форму. В BANK_USER_ROLES нет неключевых атрибутов, зависящих только от части составного ключа; таблица удовлетворяет второй нормальной форме.

Наименования ролей, валют и типов операций вынесены в справочники. В счетах не дублируются Ф.И.О. клиента и наименование валюты, в картах — владелец и остаток счета. Тем самым исключаются основные транзитивные зависимости неключевых атрибутов. Текущий остаток счета и последние четыре цифры PAN являются осознанно сохраняемыми производными значениями: первый нужен для блокировки и проверки доступности средств, вторые — для отображения карты без расшифрования полного номера.

Для идентификаторов принимается NUMBER(19) с генерацией GENERATED ALWAYS AS IDENTITY и ограничением PRIMARY KEY. Для сумм и лимитов используется NUMBER(18,2), исключающий ошибки двоичного представления десятичных сумм. Признак блокировки хранится как NUMBER(1) с ограничением значений 0 и 1, что совместимо с SQL-типами Oracle 19c. Номера счетов являются строками, поскольку могут содержать буквы и ведущие нули.

Временные отметки регистрации и проведения операций имеют тип TIMESTAMP(6) WITH TIME ZONE. Границы суточного лимита в учебной модели определяются по UTC и применяются единообразно при проведении операции и формировании отчета. expires_at хранит дату первого дня месяца после месяца окончания действия карты; карта действительна только до этой границы. Текстовые поля используют символьную семантику CHAR и кодировку базы данных AL32UTF8.

Шифротексты сохраняются в RAW; достаточность их длины проверяется с учетом UTF-8, дополнения до блока, вектора инициализации и кода аутентичности. NULL допускается только для необязательной электронной почты, даты закрытия, контекста аудита и иных явно необязательных значений. Обязательные ключи, состояния, суммы и реквизиты имеют NOT NULL.

2.4 Структура основных таблиц

Структура банковского счета приведена в таблице 2.2. При обычном открытии opening_balance и balance равны нулю. Ненулевой начальный остаток разрешается только как исходное состояние согласованного тестового набора либо при контролируемом начальном переносе данных; после создания счета opening_balance не изменяется.

Таблица 2.2 — Основные атрибуты BANK_ACCOUNTS

Атрибут	Тип Oracle	Назначение
account_id	NUMBER(19)	Первичный ключ счета
client_id	NUMBER(19)	Ссылка на владельца счета
currency_code	CHAR(3 CHAR)	Ссылка на разрешенную валюту
account_no	VARCHAR2(34 CHAR)	Уникальный номер счета
status	VARCHAR2(10 CHAR)	OPEN, BLOCKED или CLOSED
opening_balance	NUMBER(18,2)	Неизменяемый начальный остаток
balance	NUMBER(18,2)	Текущий неотрицательный остаток
opened_at; closed_at	TIMESTAMP(6) WITH TIME ZONE	Время открытия и закрытия
opened_by	NUMBER(19)	Сотрудник, открывший счет

Таблица BANK_OPERATIONS хранит только успешно проведенные операции. Отказ в выполнении отражается в аудите, а не маскируется записью с измененными остатками. Атрибуты операции приведены в таблице 2.3.

Таблица 2.3 — Основные атрибуты BANK_OPERATIONS

Атрибут	Тип Oracle	Назначение
operation_id	NUMBER(19)	Первичный ключ операции
initiator_user_id	NUMBER(19)	Пользователь, инициировавший списание
debit_account_id	NUMBER(19)	Счет списания
credit_account_id	NUMBER(19)	Счет зачисления
type_code	VARCHAR2(12 CHAR)	TRANSFER либо PAYMENT
amount	NUMBER(18,2)	Положительная сумма операции
purpose	VARCHAR2(250 CHAR)	Назначение; обязательно для PAYMENT
request_id	VARCHAR2(64 CHAR)	Идентификатор повторяемого запроса
created_at	TIMESTAMP(6) WITH TIME ZONE	Время проведения операции

Для карты полное значение PAN отделяется от отображаемого реквизита. pan_enc содержит защищенный контейнер, pan_hmac используется для контроля уникальности, pan_last4 — только для маскированного отображения. Типы RAW(128), RAW(32) и CHAR(4 CHAR) соответственно не предполагают хранения PIN, CVV или CVC. Эти значения в разрабатываемую модель не включаются.

Для профиля клиента full_name_enc, document_enc, phone_enc и email_enc содержат шифротексты. document_hmac вычисляется по нормализованному обозначению документа, включающему страну, тип и номер, с отдельным поисковым ключом. Это позволяет контролировать повторную регистрацию документа, не выполняя сравнение случайных шифротекстов.

2.5 Правила целостности и жизненный цикл объектов

Счет OPEN допускает денежные операции; BLOCKED и CLOSED — запрещает. Закрытие требует нулевого остатка и отсутствия действующих карт. Повторное открытие CLOSED не допускается.

Карта используется только при ACTIVE, открытом счете и непросроченном expires_at. Состояние EXPIRED дополнительно определяется проверкой даты.

Списание и зачисление используют одну сумму. Межтабличные условия проверяются процедурами, допустимые значения строки — CHECK-

ограничениями. Расчетный остаток сопоставляется с balance контрольным представлением.

3 Разработка объектов базы данных

3.1 Состав схемы и порядок создания объектов

Объекты прикладных данных размещаются в схеме BANK_OWNER. Владелец схемы используется для установки и сопровождения и не выступает учетной записью повседневной работы приложения. Проектируемый состав включает 12 таблиц, ограничения ключей и допустимых значений, индексы, внутренние представления и пакеты PL/SQL. Зависимые объекты создаются после объектов, на которые они ссылаются.

Сначала создаются справочники и BANK_USERS, затем таблица назначений ролей и профили клиентов и сотрудников. После этого создаются счета, карты, лимиты, операции и аудит. На следующем этапе определяются индексы и представления, внутренние пакеты безопасности и публичные процедурные интерфейсы. Завершается установка выдачей минимальных полномочий и загрузкой справочных значений.

Числовые идентификаторы генерируются средствами IDENTITY; отдельные последовательности и триггеры для копирования их значений не требуются. Изменение балансов не переносится в неявные строковые триггеры: одна публичная процедура явно управляет обоими счетами и записью операции. Это позволяет определить границы транзакции и обработку ошибки в одном месте.

3.2 Ограничения целостности и индексы

Основные декларативные ограничения представлены в таблице 3.1. Для всех обязательных полей устанавливается NOT NULL. Имена ограничений формируются с префиксами PK, FK, UQ и CK, чтобы по сообщению Oracle можно было определить нарушенное правило.

Таблица 3.1 — Основные ограничения целостности

Объект	Ограничение
BANK_USERS	Уникальность oracle_username и login; is_blocked IN (0, 1). Имена учетных записей нормализуются до проверки уникальности.
BANK_CLIENTS; BANK_EMPLOYEES	Уникальность user_id и ссылка на BANK_USERS; положительная key_version.
BANK_ACCOUNTS	Уникальность account_no; balance >= 0; opening_balance >= 0; допустимые состояния; дата закрытия задана тогда и только тогда, когда status = CLOSED.
BANK_CARDS	Уникальность pan_hmac; pan_last4 состоит из четырех цифр; состояния ACTIVE, BLOCKED, EXPIRED; положительная key_version.
BANK_OPERATIONS	amount > 0; debit_account_id <> credit_account_id; уникальность пары (initiator_user_id, request_id); непустое purpose для PAYMENT.

Объект	Ограничение
BANK_ACCOUNT_LIMITS	per_operation_limit > 0; daily_limit >= per_operation_limit; account_id является PK и FK.
Все внешние ключи	Существование родительской записи. Каскадное удаление финансовой истории не применяется.

Для ключевых и поисковых обращений предусматриваются B-tree индексы [4]. Индексы первичных и уникальных ограничений не дублируются вторыми индексами с тем же начальным набором столбцов. Для остальных внешних ключей оценивается необходимость отдельного индекса; составные индексы истории должны обслуживать отбор по счету и диапазону времени.

Таблица 3.2 — Проектируемые дополнительные индексы

Индекс	Столбцы	Назначение
IX_ACCOUNTS_CLIENT	client_id	Список счетов клиента
IX_CARDS_ACCOUNT	account_id	Карты счета и проверка закрытия
IX_OPS_DEBIT_TIME	debit_account_id, created_at	Расходная история и расчет суточного лимита
IX_OPS_CREDIT_TIME	credit_account_id, created_at	Приходная история счета
IX_USER_ROLES_ROLE	role_id	Выбор пользователей по роли
IX_AUDIT_ACTOR_TIME	actor_user_id, event_time	Аудит обращений пользователя

Индексы document_hmac и pan_hmac создаются для соответствующих UNIQUE-ограничений. Случайные шифротексты не используются как ключ поиска по открытому значению. Обоснованность дополнительных индексов должна проверяться по планам запросов и измерениям на согласованном тестовом наборе; заранее утверждать ускорение на конкретное число процентов нельзя.

3.3 Представления и процедурный интерфейс

Внутреннее представление V_ACCOUNT_HISTORY объединяет две выборки BANK_OPERATIONS через UNION ALL: списания и зачисления по счету. Для каждой записи возвращаются направление, сумма, контрагент, назначение и время. Использование UNION ALL не устраняет разные операции с совпадающими суммами и датами. Клиентская процедура дополнительно проверяет владельца указанного счета.

V_BALANCE_CONTROL сопоставляет balance со значением opening_balance + сумма зачислений − сумма списаний. V_ACCOUNT_SUMMARY содержит счет, валюту, состояние, остаток и лимиты. V_CLIENTS_MASKED и V_CARDS_MASKED формируют безопасный состав полей. Прямые права чтения этих представлений

прикладным ролям не выдаются: они используются процедурами выборки и отчетов.

Пакеты группируются по назначению. Все опубликованные точки доступа являются процедурами; вспомогательные функции допускаются только внутри реализации. Предусматривается следующий интерфейс, представленный в таблице 3.3.

Таблица 3.3 — Проектируемые пакеты и процедуры

Пакет и основные процедуры	Назначение и доступ
BANK_REGISTRATION_API register_client	Ограниченный сервис регистрации: создать заблокированную учетную запись Oracle, профиль и только роль CLIENT; активировать после согласованного завершения.
BANK_ADMIN_API create_client; create_employee; update_user; set_user_block; set_user_roles	Администратор: управление прикладными учетными записями и ролями.
BANK_ACCOUNTS_API open_account; set_account_status; close_account; get_account_limits	Сотрудник: обслуживание счетов; создание лимитов в транзакции открытия; чтение доступных остатков.
BANK_CARDS_API issue_card; set_card_status; get_cards	Сотрудник: выпуск и изменение состояния; клиент: просмотр только собственных карт через отдельную процедуру get_my_cards.
BANK_OPERATIONS_API transfer; make_payment	Клиент: провести перевод или платеж с проверкой владельца, суммы, состояния и лимитов.
BANK_READ_API get_my_accounts; get_my_cards; get_client_profile; get_statement	Разрешенное чтение с проверкой роли и владельца. Результат — SYS_REFCURSOR либо CLOB.
BANK_REPORTS_API get_client_report; get_account_report; get_operation_report	Администратор: отчеты с допустимым составом и маскированием полей.
BANK_JSON_API import_data; export_data	Уполномоченная административная роль: проверяемый импорт и разрешенный экспорт; собственная выписка клиента экспортируется через BANK_READ_API.
BANK_SECURITY; BANK_CRYPTO; BANK_AUDIT	Внутренние проверки пользователя и ролей, шифрование, маскирование и аудит. Прямое исполнение прикладным ролям запрещено.

Спецификация параметров процедуры transfer включает p_debit_account_id, p_credit_account_id, p_amount, p_purpose и p_request_id; результат возвращается через p_operation_id OUT. Идентификатор инициатора не принимается от клиента. Он определяется по SYS_CONTEXT('USERENV', 'SESSION_USER') и BANK_USERS. Перед операцией проверяются активность учетной записи и наличие роли CLIENT.

Для параметров периода используется полуоткрытый интервал [p_from, p_to). Процедура get_statement принимает идентификатор счета и границы периода, проверяет владельца и возвращает строки истории. Функции расшифрования не объявляются DETERMINISTIC и не применяются как обходной путь для функционального индекса с открытыми данными.

Пакетные привилегии Oracle действуют на пакет целиком, а не на отдельный его метод. Поэтому административные процедуры не размещаются в одном публичном пакете с доступными клиенту. Для BANK_CARDS_API служебные операции доступны сотруднику; клиентская get_my_cards относится к BANK_READ_API. Проверка роли внутри процедуры дополняет, но не заменяет разграничение EXECUTE.

3.4 Транзакционная обработка переводов и платежей

Публичная процедура проведения операции является самостоятельной единицей работы. На входе она устанавливает точку сохранения, определяет инициатора и проверяет формат суммы и request_id. Сумма с более чем двумя десятичными разрядами отклоняется, а не округляется при записи. Затем выполняется поиск ранее проведенного запроса с той же парой (initiator_user_id, request_id). Если параметры совпадают, возвращается прежний operation_id; если отличаются — выдается ошибка конфликта запроса.

Строки обоих счетов блокируются отдельными SELECT ... FOR UPDATE WAIT 5 последовательно в порядке возрастания account_id [3]. После получения блокировок повторно проверяются идемпотентность, владелец расходного счета, состояния и совпадение валют. Изменение лимитов и закрытие счета должны использовать ту же блокировку строки счета, чтобы не выполняться параллельно с проведением операции.

Сумма сравнивается с balance и per_operation_limit. Суточный расход рассчитывается по уже проведенным списаниям с debit_account_id в границах текущих суток UTC и увеличивается на сумму нового запроса. Сравнение с daily_limit выполняется при удержании блокировки расходного счета. При отсутствии строки лимитов проведение запрещается; отсутствие лимита не трактуется как неограниченный доступ.

После проверок остаток расходного счета уменьшается, остаток приходного увеличивается, в BANK_OPERATIONS добавляется одна запись и фиксируется событие успешного аудита. Только после выполнения всех действий процедура вызывает COMMIT. При исключении изменения с начала процедуры откатываются, исключение получает прикладной код через RAISE_APPLICATION_ERROR, а клиент не получает подтверждение успешного перевода.

Проведенные операции не редактируются. Исправление ошибочно проведенного движения должно оформляться отдельной компенсирующей операцией, а не изменением исторической суммы. Включение полноценного

процесса согласования возвратов является отдельным расширением проекта; произвольная отмена проведенной операции клиентом не предусматривается.

3.5 Шифрование и маскирование конфиденциальных данных

Для защищаемых полей предусматривается симметричное шифрование средствами DBMS_CRYPTO. Выбирается AES с ключом 256 бит, режимом CBC и дополнением PKCS5; исходный текст преобразуется в RAW в кодировке AL32UTF8. Для каждого шифрования генерируется новый 16-байтный IV с помощью RANDOMBYTES [1]. Одинаковые исходные значения не должны порождать одинаковые шифротексты.

CBC самостоятельно не обеспечивает проверку подлинности. Поэтому проектный контейнер предусматривает схему Encrypt-then-MAC: IV, шифротекст и HMAC-SHA-256. Код MAC рассчитывается отдельным ключом и связывает шифротекст с назначением поля, идентификатором записи и версией ключей. До расшифрования выполняется проверка MAC; некорректный контейнер отклоняется без выдачи открытого текста. Сравнение кода аутентичности не должно завершаться при первом отличающемся байте.

Для поиска и контроля уникальности используются отдельные HMAC отпечатки document_hmac и pan_hmac. Обычный несекретный хеш документа либо номера карты не применяется как достаточная защита от перебора. Поисковый ключ отделяется от ключей шифрования и проверки контейнера. Последние четыре цифры PAN сохраняются отдельно только для формирования маски и не заменяют шифрование полного номера.

Поле key_version определяет версию набора ключей шифрования и MAC. Сами ключи не включаются в исходный код пакета, прикладные таблицы или JSON. Для реализации предусматривается отдельный провайдер ключей, доступный только внутреннему криптографическому пакету на стороне сервера; конкретная интеграция с защищенным хранилищем должна быть выполнена при развертывании. DBMS_CRYPTO не рассматривается как самостоятельная система хранения и ротации ключей. При смене ключей старые версии сохраняются до завершения перешифрования и проверки резервных копий.

Для маскирования задаются правила, соответствующие характеру поля. Карта отображается как «**** **** **** 1234», телефон — с сохранением последних четырех цифр, электронная почта — с частичным скрытием локальной части, документ — с ограниченным фрагментом номера. Полный PAN не возвращается ни одной прикладной роли. Клиент получает разрешенные персональные поля собственного профиля; сотрудник — данные в объеме конкретной операции обслуживания; административные отчеты используют маскированное представление.

Функции маскирования вызываются в базе данных перед выдачей результата. Скрытие символов только в пользовательском интерфейсе недостаточно, поскольку открытое значение уже было бы передано

приложению. Шифрование полей также не заменяет защиту соединения приложения с Oracle и контроль прав владельца схемы. Доступные владельцу схемы и техническому администратору полномочия рассматриваются отдельно от бизнес-ролей.

3.6 Роли Oracle и контроль выполнения процедур

Прикладные роли сопоставляются с Oracle-ролями R_BANK_ADMIN, R_BANK_EMPLOYEE и R_BANK_CLIENT. Ролям выдаются только CREATE SESSION и EXECUTE на соответствующие публичные пакеты. Таблицы, внутренние представления и BANK_CRYPTO не доступны напрямую; у конечных пользователей отсутствуют SELECT ANY TABLE, EXECUTE ANY PROCEDURE и привилегии DBA.

Пакеты, выполняющие обращения к таблицам BANK_OWNER, проектируются с AUTHID DEFINER. Необходимые объектные привилегии владельцу выдаются непосредственно, поскольку роль не подменяет прямые полномочия в таком контексте [2]. Имя подключившегося пользователя определяется через SESSION_USER, а не CURRENT_USER, который внутри пакета с правами владельца отражает другой контекст. Произвольные параметры user_id и устанавливаемый приложением CLIENT_IDENTIFIER не признаются доказательством личности.

Создание пользователей Oracle, блокировка их входа и выдача ролей требуют отдельного ограниченного механизма подготовки учетных записей. Динамические команды допускают только проверенные идентификаторы и фиксированный набор ролей; защиту идентификаторов обеспечивает DBMS_ASSERT. Сервис регистрации имеет право только на register_client и не может назначить ADMIN или EMPLOYEE.

DDL-команды Oracle выполняют неявную фиксацию транзакции, поэтому создание пользователя не объединяется с денежной операцией. Для регистрации предусматриваются последовательные этапы: создание заблокированной Oracle-учетной записи, сохранение прикладного профиля и роли CLIENT, затем разрешение входа. При незавершенном процессе учетная запись остается заблокированной и требует завершения либо очистки административным механизмом. Изменение прикладного is_blocked проверяется каждым API, в том числе в уже открытой сессии.

3.7 Аудит и подготовка к последующим этапам

BANK_AUDIT_LOG предназначается для регистрации управления пользователями, изменения ролей и состояний объектов, денежных операций, обращения к конфиденциальным полям и выполнения импорта или экспорта. Запись содержит прикладного автора при его наличии, имя сессии Oracle, код действия, объект, время, результат и идентификатор запроса. Пароли, PAN, ключи и открытые значения документа в журнал не записываются.

Событие успешного изменения сохраняется в той же транзакции, что и изменение данных. Для события отказа допускается отдельная автономная

процедура аудита, которая не изменяет заблокированные бизнес-строки и не ссылается на еще не зафиксированного пользователя. Клиентские роли не получают права редактирования и удаления журнала. Ошибка записи обязательного успешного аудита должна приводить к отмене соответствующей бизнес-операции.

Для JSON предусматривается BANK_JSON_API: входной CLOB проверяется и разбирается средствами JSON_TABLE с явной обработкой ошибок, а выход формируется JSON_OBJECT и JSON_ARRAYAGG с RETURNING CLOB [5, 6]. Процедуры обмена используют те же проверки полномочий и маскирования, что и обычное чтение. Импорт счетов, карт и финансовых записей не должен предоставлять прямой путь изменения балансов или обхода процедур проведения.

Для дальнейшей оценки производительности формируется согласованный набор из не менее 100 000 строк BANK_OPERATIONS с корректными ссылками, датами и суммами. Подготавливаются запрос истории по счету и периоду, расчет суточного расхода и сводный отчет. После загрузки обновляется статистика оптимизатора, сохраняются планы и измеряются времена выполнения. На данном этапе определены объекты и условия испытаний; результаты импорта, расшифрования и нагрузочного тестирования должны быть получены при реализации и проверке базы данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Oracle Database PL/SQL Packages and Types Reference 19c. DBMS_CRYPTO [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/en/database/oracle/oracle-database/19/arpls/DBMS_CRYPTO.html. — Дата доступа: 27.10.2026.
2. Oracle Database PL/SQL Language Reference 19c. Invoker’s Rights and Definer’s Rights [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/invokers-rights-and-definers-rights-authid-property.html. — Дата доступа: 27.10.2026.
3. Oracle Database SQL Language Reference 19c. SELECT [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/SELECT.html. — Дата доступа: 27.10.2026.
4. Oracle Database SQL Language Reference 19c. CREATE INDEX [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/CREATE-INDEX.html. — Дата доступа: 27.10.2026.
5. Oracle Database JSON Developer’s Guide 19c. SQL/JSON Function JSON_TABLE [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/en/database/oracle/oracle-database/19/adjsn/function-JSON_TABLE.html. — Дата доступа: 27.10.2026.
6. Oracle Database JSON Developer’s Guide 19c. Generation of JSON Data with SQL/JSON Functions [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/en/database/oracle/oracle-database/19/adjsn/generation.html. — Дата доступа: 27.10.2026.